

Ajloun National University

Faculty of Information Technology

AI in Cybersecurity and Cloud Computing (314341)

Lab Manual

18 labs: no-code tools (Orange, Teachable Machine, web tools) and Python in Google Colab

Dr. Mohammed Tawfik

Head of the Department of Cyber Security and Cloud Computing

How to use this manual

- Each lab has an objective, the tools and data, numbered steps, questions and what to hand in.
- No-code labs use Orange Data Mining (free): orangedatamining.com. Install it on your laptop before Lab 2.
- Python labs run in Google Colab in the browser: nothing to install. Open the notebook from the link in the lab, sign in with a Google account, and run the cells from top to bottom.
- Kaggle datasets need a free Kaggle account to download.
- Hand in a short report for each lab: screenshots, the numbers you found, and your answers in your own words.

Safety and ethics rules

- Practise attacks only on sites made for learning (Gandalf, flaws.cloud) or on your own systems.
- Never paste real passwords, keys, or personal data of students or staff into chatbots or public tools.
- The data sets in these labs contain features or invented records only; no real malware files are used.

Lab overview

Lab	Title	Chapter	Type	Minutes
1	Meet AI: Train Your First Model	1	No code	60
2	A First Look at Security Data in Orange	2	No code	60
3	A Phishing-Website Detector with a Decision Tree	2	No code	75
4	An SMS Spam Detector that Reads Text	2	No code	75
5	A Phishing E-mail Detector in Python	2	Python (Colab)	75
6	Finding SSH Brute-Force Attacks in Real Logs	3	Python (Colab)	75
7	Preparing Intrusion Data in Orange	3	No code	60
8	Cleaning and Preparing a Login Table in Python	3	Python (Colab)	75
9	Train, Test and Judge an IDS in Orange	4	No code	75
10	Complete Python Example: An AI Intrusion Detector	4	Python (Colab)	90
11	Card Fraud and the Accuracy Trap	4	Python (Colab)	90
12	Android Malware Detection with a Random Forest	4	Python (Colab)	75
13	Inside a Neural Network	5	No code	60
14	Generative AI as a SOC Assistant	5	No code	60
15	Prompt Injection with Gandalf	5	No code	60
16	Investigating a Cloud Account with CloudTrail	6	Python (Colab)	75
17	Cloud Misconfiguration: flaws.cloud	6	No code	45
18	Threat-Modelling a University Chatbot	6	No code (group)	60

Lab 1: Meet AI: Train Your First Model

Chapter	1
Type	No code
Time	60 minutes
Tools	Google Teachable Machine (teachablemachine.withgoogle.com), a laptop with a webcam
Data	Pictures you take with the webcam

Objective: Train an image model without code, test it, and see how and why it makes mistakes.

Background: A door-access system decides from a camera picture whether to open. You will build a toy version: the model learns from examples you show it (learning with answers), then decides on new pictures.

Steps

1. Open teachablemachine.withgoogle.com → Get Started → Image Project → Standard image model.
2. Rename “Class 1” to “ID card” and “Class 2” to “No card”.
3. Click Webcam under “ID card” and record about 40 pictures of your university ID card from different angles.
4. Record about 40 pictures for “No card” (your hand, a phone, the empty desk).
5. Click Train Model and wait until it finishes.
6. Test in the Preview panel: show the ID card, then other objects. Note the percentages.
7. Show something the model never saw (a bank card, a book). What does it say?
8. Change the light (turn away from the window) and test again.
9. Add a third class “Other card”, record pictures, retrain and test again.

Questions

- Why did the model give an answer for an object it never saw?
- When did it make mistakes? Explain why.
- How is this like a security detector (for example a spam filter or face unlock)?
- What would an attacker try in order to fool this door system?

Hand in: Screenshot of the trained model in Preview, and short answers to the questions.

Lab 2: A First Look at Security Data in Orange

Chapter	2
Type	No code
Time	60 minutes
Tools	Orange Data Mining (free, orangedatamining.com)
Data	dataset_phishing.csv from Kaggle "Web page Phishing Detection" (11,430 web addresses, 87 features, column status)

Objective: Open a real security dataset, recognise examples, features and the label, and find features that separate phishing from legitimate sites.

Background: Each row is one web address. The columns describe the address and the page (length, number of dots, use of an IP address, …). The column "status" is the label: phishing or legitimate.

Steps

1. Download dataset_phishing.csv from kaggle.com/datasets/shashwatwork/web-page-phishing-detection-dataset.
2. In Orange, drag a File widget onto the canvas and open the file.
3. In the File widget set the role of "status" to target and the role of "url" to meta (it is text, not a feature).
4. Connect File → Data Table. Scroll through some rows and columns.
5. Connect File → Feature Statistics. Look at the number of columns and any missing values.
6. Connect File → Distributions. Choose length_url and split by status. Then try nb_dots and ip.
7. Connect File → Scatter Plot. Put length_url on X and nb_hyperlinks on Y, coloured by status.

Questions

- What is one row in this table? Which column is the label? Name three features.
- How many phishing and legitimate rows are there? Is the data balanced?
- Which feature looks most useful for separating phishing from legitimate sites? Show it with Distributions.
- Why should "url" not be used as a normal feature?

Hand in: Saved Orange workflow (.ows) and screenshots of two Distributions charts with your answers.

Lab 3: A Phishing-Website Detector with a Decision Tree

Chapter	2
Type	No code
Time	75 minutes
Tools	Orange Data Mining
Data	dataset_phishing.csv (Lab 2)

Objective: Train a decision tree that detects phishing websites, read the rules it learned, and use it on new addresses.

Background: A decision tree learns a chain of yes/no questions from the labelled examples. Unlike a black box, an analyst can read why it decided.

Steps

1. Start from the Lab 2 workflow (status = target, url = meta).
2. Add Data Sampler: 80% of the data (Stratify sample ticked). Its first output (Data Sample) is the training set; the second (Remaining Data) is the test set.
3. Connect Data Sample → Tree. In Tree set "Limit the maximal tree depth to" 4.
4. Connect Tree → Tree Viewer. Read the first three questions.
5. Connect Tree and Remaining Data → Predictions. Look at the tree's answers on the test rows.
6. Add Test and Score: connect Data Sample to its Data input, Remaining Data to its Test Data input and Tree to Learner. Choose "Test on test data", then connect Test and Score → Confusion Matrix.
7. Change the depth to 2 and then 10. Watch how the tree and the results change.

Questions

- Write the tree's main rule in plain English.
- Do you agree with the rule as a security person?
- How many phishing sites were missed? How many false alarms?
- What happened with depth 2 and depth 10? Which is closer to underfitting, which to overfitting?

Hand in: Screenshot of the Tree Viewer, the confusion matrix, and your answers.

Lab 4: An SMS Spam Detector that Reads Text

Chapter	2
Type	No code
Time	75 minutes
Tools	Orange Data Mining with the Text add-on (Options → Add-ons → Text)
Data	spam.csv from Kaggle "SMS Spam Collection Dataset" (5,572 messages; v1 = label, v2 = text)

Objective: Turn text messages into numbers (bag of words) and train a spam detector.

Background: Smishing (SMS phishing) is common. A model cannot read words; the bag of words counts how often each word appears, and the model learns which words point to spam.

Steps

1. Install the Text add-on: Options → Add-ons → tick Text → OK, then restart Orange.
2. Download spam.csv from kaggle.com/datasets/uciml/sms-spam-collection-dataset. If Orange cannot read it, open it in Excel and save it as "CSV UTF-8".
3. Use a Corpus widget to open the file. Set v2 as the text feature and v1 as the target. Ignore the empty extra columns.
4. Connect Corpus → Preprocess Text (lowercase, remove punctuation, remove English stop words).
5. Connect → Bag of Words. Then → Word Cloud, and compare the words of spam and ham (select only spam rows with Select Rows if needed).
6. Connect Bag of Words → Test and Score, and add Logistic Regression and Naive Bayes as learners. Use 5-fold cross-validation.
7. Connect Test and Score → Confusion Matrix.

Questions

- Which words are typical of spam?
- Which model caught more spam? Which raised fewer false alarms?
- Is the data balanced? Why does that matter for accuracy?
- Write one new SMS that you think the model would miss. Why?

Hand in: Screenshots of the Word Cloud and Confusion Matrix, and your answers.

Lab 5: A Phishing E-mail Detector in Python

Chapter	2
Type	Python (Colab)
Time	75 minutes
Tools	Google Colab, notebook Lab05_Phishing_Email_Text.ipynb
Data	Phishing_Email.csv from Kaggle “Phishing Email Detection” (about 18,650 e-mails)

Objective: Build a text classifier for phishing e-mails, read its results and find the words it relies on.

Background: TF-IDF turns each e-mail into numbers that show which words matter in it. Logistic regression then learns which words point to phishing.

Steps

1. Open the notebook: https://colab.research.google.com/github/kmkholm/ai-cybersecurity-cloud-course/blob/main/labs/Lab05_Phishing_Email_Text.ipynb
2. Download Phishing_Email.csv from Kaggle and upload it with the Files panel in Colab.
3. Run the cells one by one and read the comments.
4. Record the confusion matrix and the precision and recall of “Phishing Email”.
5. Look at the words that point to phishing and to safe e-mail.
6. In the last cell write two e-mails of your own and check the model’s answers.

Questions

- How many phishing e-mails were caught and how many missed?
- Why is the vocabulary learned from the training e-mails only?
- Do the phishing words make sense? Give three.
- Write a phishing e-mail the model labels as safe. What does this teach about attackers?

Hand in: Notebook link or PDF with outputs, and your answers.

Lab 6: Finding SSH Brute-Force Attacks in Real Logs

Chapter	3
Type	Python (Colab)
Time	75 minutes
Tools	Google Colab, notebook Lab06_SSH_Brute_Force_Logs.ipynb
Data	Loghub OpenSSH_2k (2,000 real SSH log lines, parsed); loaded from GitHub

Objective: Process real logs into features: extract addresses and user names, count failed logins per address, and raise alerts.

Background: This is steps 2 to 5 of Chapter 3 on real data: parse, select, build features in groups, then a first alarm rule.

Steps

1. Open the notebook: https://colab.research.google.com/github/kmkholm/ai-cybersecurity-cloud-course/blob/main/labs/Lab06_SSH_Brute_Force_Logs.ipynb
2. Run section 1 and look at the columns Content and EventTemplate.
3. Run section 2: which message templates are most common?
4. Run sections 3–4: build the table of failed logins and users tried per address.
5. Run section 5 with LIMIT = 10, then LIMIT = 3 and LIMIT = 30.
6. Run section 6: did any alerted address log in successfully?

Questions

- How many failed password lines are there? From how many addresses?
- Which address made the most attempts? How many user names did it try?
- What happens to the number of alerts when LIMIT goes from 30 to 3? Link this to the alarm limit (Chapter 4).
- Name two more features that would help detect brute force.

Hand in: Notebook with outputs and your answers.

Lab 7: Preparing Intrusion Data in Orange

Chapter	3
Type	No code
Time	60 minutes
Tools	Orange Data Mining
Data	Train_data.csv from Kaggle “Network Intrusion Detection” (25,192 connections, 41 features, class = normal / anomaly)

Objective: Check data quality, turn categories into numbers, scale numbers and split the data with a stratified sample.

Background: Steps 4, 6 and 8 of Chapter 3 in a no-code tool: Feature Statistics (quality), Preprocess (one-hot and scaling), Data Sampler (split).

Steps

1. Open Train_data.csv in a File widget; set class as the target.
2. Connect → Feature Statistics: missing values? Which columns have the widest range?
3. Connect File → Preprocess. Add “Continuize Discrete Variables” (one feature per value) and “Normalize Features” (interval 0–1).
4. Connect File → Data Table and Preprocess → Data Table. Compare the number of columns before and after.
5. Connect Preprocess → Data Sampler: 70%, Stratify sample ticked.
6. Connect both outputs of Data Sampler to Distributions on class: is the share of anomaly the same?

Questions

- Which three columns are text categories? How many columns after one-hot encoding? Why more?
- Why do we scale before using k-NN?
- Is the share of anomaly the same in both parts? What does “stratify” do?
- Why should scaling be learned from the training part only in a real project?

Hand in: Workflow (.ows), screenshots of both Data Tables and the two Distributions, and your answers.

Lab 8: Cleaning and Preparing a Login Table in Python

Chapter	3
Type	Python (Colab)
Time	75 minutes
Tools	Google Colab, notebook Lab08_Cleaning_Login_Data.ipynb
Data	dirty_logins.csv (made for this course, loaded from GitHub)

Objective: Find and fix data problems, keep the attacks, convert time to UTC, encode categories and split by time.

Background: The table has mistakes on purpose: duplicates, missing countries, an impossible value, mixed spellings and outliers that are real attacks.

Steps

1. Open the notebook: https://colab.research.google.com/github/kmkholm/ai-cybersecurity-cloud-course/blob/main/labs/Lab08_Cleaning_Login_Data.ipynb
2. Section 1: list every problem you see.
3. Section 2: clean the data. Check the size and the country values after cleaning.
4. Section 3: convert to UTC and build hour and night features.
5. Section 4: look at the biggest failed_attempts values before deleting anything.
6. Sections 5–6: one-hot encode, split by time, train a small tree and read its rule.

Questions

- How many rows, duplicates, missing countries and impossible values were there?
- Why do we keep the row with a missing country when it is an attack?
- Are the large failed_attempts values errors or attacks? How do you know?
- Why split by time instead of randomly?

Hand in: Notebook with outputs and your answers.

Lab 9: Train, Test and Judge an IDS in Orange

Chapter	4
Type	No code
Time	75 minutes
Tools	Orange Data Mining
Data	Train_data.csv (Lab 7)

Objective: Train two models, read their confusion matrices, and calculate detection rate and precision by hand.

Background: Test and Score trains and tests learners in one widget. The Confusion Matrix widget shows the four outcomes of Chapter 4.

Steps

1. File (class = target) → Test and Score. Add Tree and kNN as learners.
2. In Test and Score choose Random sampling, training set size 70%, 1 repetition.
3. Read CA, Precision, Recall and F1 for each model.
4. Connect Test and Score → Confusion Matrix. Select each model in turn.
5. From each matrix calculate by hand: accuracy, detection rate of anomaly, precision of anomaly.
6. Set “Target class” to anomaly in Test and Score and check that your numbers match.

Questions

- Write both confusion matrices.
- Show your calculations of detection rate and precision for anomaly.
- Which model would you choose for the university network? Why?
- Why can accuracy alone be misleading for intrusion detection?

Hand in: Screenshots of Test and Score and both Confusion Matrices, with calculations by hand.

Lab 10: Complete Python Example: An AI Intrusion Detector

Chapter	4
Type	Python (Colab)
Time	90 minutes
Tools	Google Colab, notebook Chapter4_Complete_Example.ipynb
Data	Train_data.csv (Lab 7), uploaded to Colab

Objective: Run the complete example of Chapter 4: prepare, train, test, measure, read the tree, change the alarm limit and compare with k-NN.

Background: The slides of Chapter 4 explain every line of this notebook with numbered call-outs.

Steps

1. Open the notebook: https://colab.research.google.com/github/kmkholm/ai-cybersecurity-cloud-course/blob/main/labs/Chapter4_Complete_Example.ipynb
2. Upload Train_data.csv with the Files panel.
3. Run steps 1–7 and write down the confusion matrix and the report.
4. Run Extra 1 (depths), Extra 2 (rules), Extra 3 (alarm limit), Extra 4 (k-NN).
5. Calculate detection rate and precision by hand from your matrix.

Questions

- Your confusion matrix and your hand calculations.
- What do the depth results show about overfitting?
- Which alarm limit would you choose and why?
- Which model is better on your run: the tree or k-NN?

Hand in: Notebook with all outputs and your answers.

Lab 11: Card Fraud and the Accuracy Trap

Chapter	4
Type	Python (Colab)
Time	90 minutes
Tools	Google Colab, notebook Lab11_Fraud_Imbalanced_Data.ipynb
Data	creditcard.csv from Kaggle “Credit Card Fraud Detection” (284,807 payments, 492 frauds)

Objective: See why accuracy misleads on rare attacks, and improve the detection of frauds with class weights and the alarm limit.

Background: Only 0.172% of the payments are frauds. A model can be 99.8% “accurate” and useless.

Steps

1. Open the notebook: https://colab.research.google.com/github/kmkholm/ai-cybersecurity-cloud-course/blob/main/labs/Lab11_Fraud_Imbalanced_Data.ipynb
2. Upload creditcard.csv (about 150 MB) to Colab.
3. Run section 2 (lazy model) and write down accuracy and frauds caught.
4. Run sections 3 and 4 and compare detection rate and precision.
5. Run section 5 and fill a table: limit, caught, false alarms.

Questions

- Accuracy and frauds caught of the lazy model. What does this prove?
- What changed with class_weight="balanced"? What did it cost?
- Which alarm limit would a bank choose? Why?
- Why must the test set never be balanced?

Hand in: Notebook with outputs, the limit table and your answers.

Lab 12: Android Malware Detection with a Random Forest

Chapter	4
Type	Python (Colab)
Time	75 minutes
Tools	Google Colab, notebook Lab12_Android_Malware.ipynb
Data	drebin-215-dataset-5560malware-9476-benign.csv from Kaggle “Android Malware Dataset for Machine Learning” (15,036 apps)

Objective: Compare one decision tree with a random forest and find the permissions and API calls that point to malware.

Background: Each app is described by 215 yes/no features: permissions it asks for and API calls it uses. No real malware is included.

Steps

1. Open the notebook: https://colab.research.google.com/github/kmkholm/ai-cybersecurity-cloud-course/blob/main/labs/Lab12_Android_Malware.ipynb
2. Upload the CSV file to Colab.
3. Run section 1: which class value is malware?
4. Run sections 2 and 3 and compare the two reports.
5. Run section 4 and look up three of the top features (what does the permission allow?).

Questions

- Which label means malware? How did you know?
- Compare the tree and the forest: detection rate and precision for malware.
- Explain why 100 trees voting can be better than one tree.
- Choose one top feature and explain why malware might use it.

Hand in: Notebook with outputs and your answers.

Lab 13: Inside a Neural Network

Chapter	5
Type	No code
Time	60 minutes
Tools	TensorFlow Playground (playground.tensorflow.org), Orange Data Mining
Data	Playground data sets; Train_data.csv (Lab 7)

Objective: Watch a neural network learn, see why hidden layers matter, and compare a neural network with a decision tree.

Background: A neural network adjusts its weights step by step (backpropagation). More layers can learn more complex shapes.

Steps

1. Open playground.tensorflow.org. Choose the “Circle” data set, 0 hidden layers, and press Run. Watch the test loss.
2. Add one hidden layer with 4 neurons and run again.
3. Choose the “Spiral” data set. Try 2 hidden layers with 6 neurons each.
4. Increase the noise to 30. What happens to the test loss?
5. In Orange: File (Train_data.csv) → Test and Score with Tree and Neural Network learners.

Questions

- Why can a network with no hidden layer not separate the circle?
- What did the hidden layers learn (look at the neuron pictures)?
- Tree or neural network on the intrusion data: which is better, and which is easier to explain?
- Why is a neural network called a “black box”?

Hand in: Screenshots of two Playground runs and of Test and Score, with your answers.

Lab 14: Generative AI as a SOC Assistant

Chapter	5
Type	No code
Time	60 minutes
Tools	Any free chatbot (ChatGPT, Gemini, Copilot or Claude); nvd.nist.gov to check CVEs
Data	The invented e-mail and log lines printed below

Objective: Write good prompts for security tasks and check the answers for mistakes (hallucinations).

Background: LLMs help analysts read and write faster, but they can be confidently wrong. Never paste real passwords, keys or personal data.

E-mail for this lab (invented)

```
From: IT Support <it-support@ajloun-nationa1.com>
Reply-To: helpdesk@mail-box.co
Subject: URGENT: your password expires today

Dear staff member, your university password will expire within 24 hours. To keep access to
your e-mail and salary page, confirm your account here:
http://203.0.113.45/anu-login/verify.php
Attachment: salary_update.zip
```

Log lines for this lab (invented)

```
Nov 03 02:28:01 srv1 sshd: Failed password for root from 203.0.113.45 port 51522 ssh2
Nov 03 02:28:09 srv1 sshd: Failed password for admin from 203.0.113.45 port 51530 ssh2
Nov 03 02:28:15 srv1 sshd: Failed password for test from 203.0.113.45 port 51544 ssh2
Nov 03 02:28:41 srv1 sshd: Accepted password for test from 203.0.113.45 port 51601 ssh2
Nov 03 02:31:02 srv1 sshd: session opened for user test
Nov 03 02:33:40 srv1 sudo: test : COMMAND=/bin/cat /etc/shadow
```

Steps

1. Prompt A (vague): paste the e-mail below and write only “is this ok?”. Save the answer.
2. Prompt B (good): give a role, a task, the data and the format (Chapter 5): “You are a SOC analyst. List the phishing clues in this e-mail as bullet points and give a verdict.”
3. Paste the log lines below and ask for a short incident summary and the next three steps.
4. Ask: “Write a Microsoft Sentinel KQL query to find more than 50 failed Windows logons from one IP address in the last hour.”
5. Ask: “Which CVE does this attack exploit?” Then look up the CVE number it gives on nvd.nist.gov.
6. Mark every statement you could not verify.

Questions

- Compare the answers to prompt A and prompt B. Why is B better?
- Was the incident summary correct? What did it miss or add?
- Did the CVE exist? Was it related? What does this show?
- Write three rules for using chatbots in the SOC.

Hand in: Screenshots or copies of the four conversations, with your marks and answers.

Lab 15: Prompt Injection with Gandalf

Chapter	5
Type	No code
Time	60 minutes
Tools	Gandalf by Lakera (gandalf.lakera.ai)
Data	None

Objective: Experience prompt injection safely and link each defence to the OWASP Top 10 for LLM Applications.

Background: Gandalf is an LLM that guards a password. Each level adds a defence. You try to make it reveal the password.

Steps

1. Open gandalf.lakera.ai and read the level description.
2. Try to get the password for level 1, then continue level by level (at least to level 4).
3. For each level write: the kind of trick that worked (asking directly, role-play, spelling it out, translation, …) and what defence the level added.
4. Do not share the passwords; share only the kind of trick.

Questions

- Which kinds of tricks worked at which level?
- What defences did the higher levels add?
- Which OWASP risks does this game show (LLM01, LLM07)?
- Why is prompt injection hard to fix completely?

Hand in: A table: level, trick type, defence added; and your answers.

Lab 16: Investigating a Cloud Account with CloudTrail

Chapter	6
Type	Python (Colab)
Time	75 minutes
Tools	Google Colab, notebook Lab16_CloudTrail_Investigation.ipynb
Data	cloudtrail_events.json (made for this course, loaded from GitHub)

Objective: Read cloud audit logs, find unusual activity and write the attack timeline.

Background: Something happened in the university cloud account on the night of 3 November. Field names are as in AWS CloudTrail; the events are invented.

Steps

1. Open the notebook: https://colab.research.google.com/github/kmkholm/ai-cybersecurity-cloud-course/blob/main/labs/Lab16_CloudTrail_Investigation.ipynb
2. Section 2: describe what is normal (users, address, region).
3. Section 3: list the unusual events.
4. Section 4: count what the attacker did and note the first and last time.
5. Section 5: decide whether every night event is an attack.

Questions

- Which account was used by the attacker, from which address?
- Write the attack timeline (UTC) in 5–6 steps.
- What was the attacker trying to achieve with RunInstances, CreateUser, AttachUserPolicy and CreateAccessKey?
- Is the night activity of backup-svc at 21:00 UTC an attack? Why not?
- What should the SOC do in the first 10 minutes?

Hand in: Notebook with outputs, your timeline and your answers.

Lab 17: Cloud Misconfiguration: flaws.cloud

Chapter	6
Type	No code
Time	45 minutes
Tools	A web browser; flaws.cloud
Data	None

Objective: See how a public storage bucket exposes files, on a site built for learning.

Background: flaws.cloud is a free learning game about common AWS mistakes, made by its author for training. Only this site may be tested; never try this on real systems.

Steps

1. Open http://flaws.cloud and read the introduction.
2. Hint: the site itself is hosted in an Amazon S3 bucket. What happens if you visit the bucket's own address instead of the web address?
3. Find the file that leads to level 2.
4. Write down which setting made this possible and how to fix it.

Questions

- What was wrong in the bucket settings?
- Which part of the shared responsibility model failed: provider or customer?
- Which AI or posture tool would have warned about it?

Hand in: A short report: what you found, the mistake, the fix.

Lab 18: Threat-Modelling a University Chatbot

Chapter	6
Type	No code (group)
Time	60 minutes
Tools	OWASP Top 10 for LLM Applications (genai.owasp.org)
Data	The scenario below

Objective: Find the main attacks on an LLM application and design the controls before it is built.

Background: Scenario: the university plans a chatbot for students. It answers questions about courses, reads files students upload, looks up the student's marks in the database, and can e-mail a transcript.

Steps

1. In groups of three, draw the parts: student, chatbot app, LLM, documents, marks database, e-mail tool.
2. For each part, choose the OWASP risks that apply (at least five different risks in total).
3. For each risk write one realistic attack on this chatbot.
4. For each attack write one control.
5. Present your table in 3 minutes.

Questions

- Fill the table: part · OWASP risk · attack · control.
- Which single control removes the most risk?
- Which actions must always need a human or a confirmation?

Hand in: The completed threat-model table (one per group).